

c0a25046bf / ProjExD_Group14

🔍

🔒

▼

+

🕒

🔗

📄

📧

<> Code

🕒 Issues

🔗 Pull requests 1

🔗 Agents

🔗 Actions

📁 Projects

📖 Wiki

🛡 Security and quality

📄 Insi

🔔

On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. [Review this update](#) and manage your preferences in your [GitHub account settings](#).

✕

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

🔗

base: main ▼

⏪

compare: C0A25046/ステージギミック ▼

✓ Able to merge. These branches can be automatically merged.

🔗 C0A25046/ステージギミック #1

No description available

🔗 View pull request

🔗 2 commits

📄 1 file changed

👤 1 contributor

🔗

Commits on May 19, 2026

ギミック追加

🔗 2553f8d <>

c0a25046bf committed 1 hour ago

ギミックを1層1つに変更

🔗 7e23673 <>

c0a25046bf committed 35 minutes ago

📄 Showing 1 changed file with 106 additions and 48 deletions.

Split Unified

▼ 🔗 154	📄	koukaton_up.py	📄
12	12		
13	13	BLACK = (20, 20, 25)	
14	14	BLUE = (100, 150, 255)	
15		- GREEN = (120, 120, 120)	
16		- RED = (255, 100, 100)	
17	15	WHITE = (255, 255, 255)	
18		- GOLD = (255, 215, 0) # ゴールブロック用の金色	
	16	+ GOLD = (255, 215, 0)	
	17	+	
	18	+ # ギミック床の色定義を復活	
	19	+ COLOR_NORMAL = (120, 120, 120) # 緑 (通常)	
	20	+ COLOR_FAKE = (180, 0, 180) # 紫 (すり抜ける)	
	21	+ COLOR_ICE = (100, 200, 255) # 水色 (滑る)	
	22	+ COLOR_TRAMPOLINE = (255, 140, 0) # オレンジ (大ジャンプ)	
	23	+ COLOR_TRAP = (255, 50, 50) # 赤 (最初に戻る)	
	24	+ COLOR_RED_UI = (255, 100, 100) # UI用赤	
	25	+	
	26	+ # 種類から色を取得する辞書	
	27	+ COLOR_MAP = {	
	28	+ "normal": COLOR_NORMAL,	
	29	+ "fake": COLOR_FAKE,	
	30	+ "ice": COLOR_ICE,	
	31	+ "trampoline": COLOR_TRAMPOLINE,	
	32	+ "trap": COLOR_TRAP	
	33	+ }	
19	34		
20	35	# =====	
21	36	# 1. データ構造 (クラス) の定義	

36	51		self._max_charge = 20.0
37	52		self._direction = 1
38	53		
39		-	self._is_clear = False # ゴールしたかどうかのフラグ
	54	+	self._is_clear = False
	55	+	self._on_ice = False
	56	+	self._is_reset = False
40	57		
41	58		# --- ゲッター ---
42	59		def get_image(self): return self._image
49	66		def get_max_charge(self): return self._max_charge
50	67		def get_direction(self): return self._direction
51	68		def get_is_clear(self): return self._is_clear
	69	+	def get_on_ice(self): return self._on_ice
	70	+	def get_is_reset(self): return self._is_reset
52	71		
53	72		# --- セッター ---
54	73		def set_vel_x(self, value): self._vel_x = value
58	77		def set_charge_power(self, value): self._charge_power = value
59	78		def set_direction(self, value): self._direction = value
60	79		def set_is_clear(self, value): self._is_clear = value
	80	+	def set_on_ice(self, value): self._on_ice = value
	81	+	def set_is_reset(self, value): self._is_reset = value
61	82		
62	83		
63	84		class Platform:
64		-	# 足場ごとに色を変えられるように引数追加
65		-	def __init__(self, x, y, w, h, color):
	85	+	def __init__(self, x, y, w, h, color, plat_type="normal"):
66	86		self._image = pygame.Surface((w, h))
67	87		self._image.fill(color)
68	88		self._rect = self._image.get_rect(topleft=(x, y))
	89	+	self._type = plat_type
69	90		
70	91		def get_image(self): return self._image
71	92		def get_rect(self): return self._rect
	93	+	def get_type(self): return self._type
72	94		
73	95		
74	96		# =====
75	97		# 2. クラスを操作する関数
76	98		# =====
77	99		
78	100		def update_player(player, keys, platforms, goal_block):
79		-	"""プレイヤーの物理挙動と状態を更新する関数"""
80		-	
81		-	# ゴール済みの場合は入力を受け付けない（重力で落ちるだけ）
82	101		if not player.get_is_clear():
83		-	# 1. 地面にいる時の処理
84	102		if player.get_on_ground():
85		-	player.set_vel_x(0)
	103	+	
	104	+	if player.get_on_ice():
	105	+	player.set_vel_x(player.get_vel_x() * 0.96)
	106	+	else:
	107	+	player.set_vel_x(0)
86	108		
87	109		if keys[pygame.K_LEFT] or keys[pygame.K_a]:
88	110		player.set_direction(-1)
105	127		player.set_is_charging(False)
106	128		player.set_charge_power(0)
107	129		player.set_on_ground(False)
	130	+	player.set_on_ice(False)
108	131		
109		-	# 2. 重力の適用
110	132		if not player.get_on_ground():
111	133		player.set_vel_y(player.get_vel_y() + GRAVITY)

112	134	
113	135	rect = player.get_rect()
114	136	
115		- # 座標の更新 (X軸) と壁の跳ね返り
116	137	rect.x += player.get_vel_x()
117	138	if rect.left < 0:
118	139	rect.left = 0
123	144	player.set_vel_x(player.get_vel_x() * -0.5)
124	145	player.set_direction(-1)
125	146	
126		- # 座標の更新 (Y軸)
127	147	rect.y += int(player.get_vel_y())
128	148	
129		- # --- 3. ゴール (天井) との当たり判定 ---
130	149	goal_rect = goal_block.get_rect()
131	150	if rect.colliderect(goal_rect):
132		- # 上昇中に天井の底にぶつかったらゴール
133	151	if player.get_vel_y() < 0:
134	152	rect.top = goal_rect.bottom
135	153	player.set_vel_y(0)
136	154	player.set_is_clear(True)
137	155	
138		- # --- 4. 足場との当たり判定 ---
139	156	player.set_on_ground(False)
	157	+ player.set_on_ice(False)
	158	+
140	159	if player.get_vel_y() > 0:
141	160	for plat in platforms:
	161	+ if plat.get_type() == "fake":
	162	+ continue
	163	+
142	164	plat_rect = plat.get_rect()
143	165	if rect.colliderect(plat_rect):
144	166	if rect.bottom <= plat_rect.top + player.get_vel_y() + 1:
	167	+
	168	+
	169	+ if plat.get_type() == "trampoline":
	170	+ rect.bottom = plat_rect.top
	171	+ player.set_vel_y(-25)
	172	+ player.set_on_ground(False)
	173	+ break
	174	+
	175	+ if plat.get_type() == "trap":
	176	+ rect.center = (WIDTH // 2, HEIGHT - 100)
	177	+ player.set_vel_y(0)
	178	+ player.set_vel_x(0)
	179	+ player.set_is_reset(True)
	180	+ break
	181	+
145	181	rect.bottom = plat_rect.top
146	182	player.set_vel_y(0)
147		- player.set_vel_x(0)
148	183	player.set_on_ground(True)
	184	+
	185	+
	186	+ if plat.get_type() == "ice":
	187	+ player.set_on_ice(True)
	188	+ else:
	189	+ player.set_vel_x(0)
	190	break
150	190	
151	191	
156	196	def main():
157	197	pygame.init()
158	198	screen = pygame.display.set_mode((WIDTH, HEIGHT))
159		- pygame.display.set_caption("Jump King - 10 Floors to Goal")
	199	+ pygame.display.set_caption("Jump King - Colored Gimmicks")
160	200	clock = pygame.time.Clock()
161	201	font = pygame.font.SysFont(None, 36)

```

162 | 202 | large_font = pygame.font.SysFont(None, 72)
163 | 203 |
164 | 204 | player = Player()
165 | - |
166 | - | # --- 10層分のマップ生成 ---
167 | 205 | platforms = []
168 | - | platforms.append(Platform(0, HEIGHT - 40, WIDTH, 40, GREEN)) # スタート床
169 | 206 | +
170 | 207 | + platforms.append(Platform(0, HEIGHT - 40, WIDTH, 40, COLOR_NORMAL, "normal"))
171 | 208 |
172 | 209 | TOTAL_FLOORS = 10
173 | - |
174 | - | # ゴール（天井）のY座標：スタート床から 10層分(800px * 10) 上へ
175 | - | goal_block = Platform(0, goal_y, WIDTH, 40, GOLD)
176 | - |
177 | - | # ゴールに到達するまで足場を生成し続ける
178 | - | platform_y = HEIGHT - 150
179 | - | while platform_y > goal_y + 100:
180 | - |     w = random.randint(60, 130)
181 | - |     x = random.randint(0, WIDTH - w)
182 | - |     platforms.append(Platform(x, platform_y, w, 20, GREEN))
183 | - |     gap_y = random.randint(80, 130)
184 | - |     platform_y -= gap_y
185 | 211 | + goal_block = Platform(0, goal_y, WIDTH, 40, GOLD, "goal")
186 | 212 | +
187 | 213 | + for f in range(TOTAL_FLOORS):
188 | 214 | +     floor_bottom = HEIGHT - (f * HEIGHT)
189 | 215 | +     floor_top = HEIGHT - ((f + 1) * HEIGHT)
190 | 216 | +
191 | 217 | +     floor_plats = []
192 | 218 | +     plat_y = floor_bottom - 150
193 | 219 | +
194 | 220 | +     while plat_y > floor_top + 50:
195 | 221 | +         w = random.randint(60, 130)
196 | 222 | +         x = random.randint(0, WIDTH - w)
197 | 223 | +         floor_plats.append(Platform(x, plat_y, w, 20, COLOR_NORMAL, "normal"))
198 | 224 | +         gap_y = random.randint(80, 130)
199 | 225 | +         plat_y -= gap_y
200 | 226 | +
201 | 227 | +         if len(floor_plats) >= 2:
202 | 228 | +             special_type = random.choice(["fake", "ice", "trampoline"])
203 | 229 | +
204 | 230 | +             # 10階の場合：ランダム特殊床1つ + トラップ床1つ
205 | 231 | +             if f == TOTAL_FLOORS - 1:
206 | 232 | +                 specials = random.sample(range(len(floor_plats)), 2)
207 | 233 | +
208 | 234 | +                 p_spec = floor_plats[specials[0]]
209 | 235 | +                 # 種類に応じた色 (COLOR_MAP[special_type]) を適用
210 | 236 | +                 floor_plats[specials[0]] = Platform(p_spec.get_rect().x, p_spec.get_rect().y,
211 | p_spec.get_rect().width, 20, COLOR_MAP[special_type], special_type)
212 | 237 | +
213 | 238 | +                 p_trap = floor_plats[specials[1]]
214 | 239 | +                 # 赤色を適用
215 | 240 | +                 floor_plats[specials[1]] = Platform(p_trap.get_rect().x, p_trap.get_rect().y,
216 | p_trap.get_rect().width, 20, COLOR_TRAP, "trap")
217 | 241 | +
218 | 242 | +             # 1～9階の場合：ランダム特殊床1つ
219 | 243 | +             else:
220 | 244 | +                 special_idx = random.choice(range(len(floor_plats)))
221 | 245 | +                 p_spec = floor_plats[special_idx]
222 | 246 | +                 floor_plats[special_idx] = Platform(p_spec.get_rect().x, p_spec.get_rect().y,
223 | p_spec.get_rect().width, 20, COLOR_MAP[special_type], special_type)
224 | 247 | +
225 | 248 | +             platforms.extend(floor_plats)
226 | 249 |

```

186	250	camera_y = 0
187	251	max_height = 0
194	258	
195	259	keys = pygame.key.get_pressed()
196	260	
197	-	# 関数呼び出し (ゴールブロックも渡して判定させる)
198	261	update_player(player, keys, platforms, goal_block)
199	262	
200	263	p_rect = player.get_rect()
201	264	
202	-	# --- カメラ処理 ---
	265	+ if player.get_is_reset():
	266	+ camera_y = 0
	267	+ player.set_is_reset(False)
	268	+
203	269	SCROLL_TOP_MARGIN = 200
204	270	if p_rect.y - camera_y < SCROLL_TOP_MARGIN:
205	271	camera_y = p_rect.y - SCROLL_TOP_MARGIN
213	279	if camera_y > 0:
214	280	camera_y = 0
215	281	
216	-	# --- スコア・階層計算 ---
217	282	current_height = (HEIGHT - 40) - p_rect.bottom
218	283	if current_height > max_height:
219	284	max_height = current_height
220	285	
221	-	# 今いる階層 (800px を 1階 とする)
222	286	current_floor = min((current_height // HEIGHT) + 1, TOTAL_FLOORS)
	287	+ display_floor = max(1, current_floor)
223	288	
224	-	# --- 描画処理 ---
225	289	screen.fill(BLACK)
226	290	
227	-	# ゴールの描画
228	291	goal_rect = goal_block.get_rect()
229	292	goal_draw_y = goal_rect.y - camera_y
230	293	if -100 < goal_draw_y < HEIGHT + 100:
231	294	screen.blit(goal_block.get_image(), (goal_rect.x, goal_draw_y))
232	295	
233	-	# 足場の描画
234	296	for plat in platforms:
235	297	plat_rect = plat.get_rect()
236	298	draw_y = plat_rect.y - camera_y
237	299	if -50 < draw_y < HEIGHT + 50:
238	300	screen.blit(plat.get_image(), (plat_rect.x, draw_y))
239	301	
240	-	# プレイヤーの描画
241	302	player_draw_y = p_rect.y - camera_y
242	303	screen.blit(player.get_image(), (p_rect.x, player_draw_y))
243	304	
244	-	# チャージゲージと方向指示器
245	305	if player.get_is_charging() and not player.get_is_clear():
246	306	gauge_width = (player.get_charge_power() / player.get_max_charge()) * 40
247	-	pygame.draw.rect(screen, RED, (p_rect.centerx - 20, player_draw_y - 15, gauge_width, 8))
	307	+ pygame.draw.rect(screen, COLOR_RED_UI, (p_rect.centerx - 20, player_draw_y - 15, gauge_width, 8))
248	308	
249	309	arrow_x = p_rect.right + 5 if player.get_direction() == 1 else p_rect.left - 10
250	310	pygame.draw.rect(screen, WHITE, (arrow_x, player_draw_y + 15, 5, 5))
251	311	
252	-	# 情報表示 (UI)
253	-	ui_text = font.render(f"Floor: {current_floor} / {TOTAL_FLOORS} Max: {max_height // 10}m", True, WHITE)
	312	+ ui_text = font.render(f"Floor: {display_floor} / {TOTAL_FLOORS} Max: {max_height // 10}m", True, WHITE)
254	313	screen.blit(ui_text, (10, 10))
255	314	

256		-	# ゴールした時の演出
257	315		if player.get_is_clear():
258	316		clear_text = large_font.render("CLEAR!!", True, GOLD)
259	317		text_rect = clear_text.get_rect(center=(WIDTH // 2, HEIGHT // 2 - 50))